

EduBench-Local: Complete Implementation Guide

Comparative Performance Analysis of Open-Source LLMs on Educational Question Answering

This guide walks through every step of the project end-to-end, in the order you should actually do them, with every tool/dataset link included and copy-pasteable commands.

Phase 0: Before You Start

Decide your compute setup first — this determines everything downstream.

Setup	Requirement	Best for
Laptop, CPU only	8 GB+ RAM	1–3B models, slower
Laptop, GPU	NVIDIA, 6 GB+ VRAM	7B models comfortably
Google Colab (free)	Free T4 GPU, ~12 hr session limit	Best free option overall
Kaggle Notebooks	Free P100/T4, 30 hrs/week	Backup if Colab quota runs out

- Google Colab: <https://colab.research.google.com>
- Kaggle Notebooks: <https://www.kaggle.com/code>

If you only have ≤8 GB RAM and no GPU, swap the 7B model later for `qwen2.5:1.5b` or `tinyllama`.

Phase 1: Environment Setup (Week 1)

Step 1 — Install Python 3.10+

Confirm with `python3 --version`. If missing, get it from <https://www.python.org/downloads/>

Step 2 — Create a virtual environment

```
bash

python -m venv edubench-env
source edubench-env/bin/activate      # Windows: edubench-env\Scripts\activate
```

Step 3 — Install core libraries

```
bash
```

```
pip install pandas numpy matplotlib seaborn
pip install rouge-score bert-score evaluate
pip install datasets transformers accelerate torch
pip install ollama
pip install streamlit # optional, for the Week 7 dashboard
```

Step 4 — Install Ollama

Ollama is what runs the LLMs locally with a single command.

- Site: <https://ollama.com>
- Docs/download: <https://ollama.com/download>

```
bash

# Mac/Linux
curl -fsSL https://ollama.com/install.sh | sh
# Windows: use the installer from https://ollama.com/download
```

Step 5 — Pull the five models under comparison

```
bash

ollama pull llama3.2:3b
ollama pull mistral:7b
ollama pull gemma2:2b
ollama pull phi3:mini
ollama pull qwen2.5:3b
```

Each pull can take several minutes depending on your connection — run this early.

Step 6 — Literature review (parallel task this week)

Read 5-8 related papers to write your Related Work section later:

- Look up: **MMLU**, **HELM**, and educational NLP benchmark papers on Google Scholar or arXiv (<https://arxiv.org>)
-

Phase 2: Dataset Preparation (Week 2)

Step 7 — Get familiar with the datasets

Dataset	What it is	Link
SciQ	Science exam Qs with ground-truth answers	https://huggingface.co/datasets/allenai/sciq
RACE	Middle/high school reading comprehension	https://huggingface.co/datasets/race
OpenBookQA (optional)	Elementary science reasoning	https://huggingface.co/datasets/openbookqa
ARC (optional)	Grade-school science	https://huggingface.co/datasets/ai2_arc
SQuAD v1.1 (optional)	General reading comprehension	https://huggingface.co/datasets/squad

General Hugging Face Datasets docs: <https://huggingface.co/docs/datasets>

Recommended combination: SciQ (science) + RACE (reading comprehension) — two different educational domains, which is your cross-domain novelty angle.

Step 8 — Load and sample the data

```
python

from datasets import load_dataset

sciq = load_dataset("allenai/sciq", split="test")
race = load_dataset("race", "middle", split="test")

sciq_sample = sciq.shuffle(seed=42).select(range(150))
race_sample = race.shuffle(seed=42).select(range(150))
```

150–300 questions per subject is enough for statistically meaningful results while keeping inference time manageable.

Step 9 — Standardize into one schema

Convert both datasets into a common structure before moving on:

```
{id, subject, question, context, reference_answer}
```

Save this as `dataset_sample.json` — this is your first deliverable.

Step 10 — Reserve a held-out slice

Set aside ~50 questions now for the Week 5 human-validation step (used to check the LLM-judge against real human ratings).

Phase 3: Answer Generation Pipeline (Week 3)

Step 11 — Build the fixed prompt template

Using one identical prompt across all models isolates model capability from prompt-engineering skill.

```
python

PROMPT_TEMPLATE = """You are a helpful and knowledgeable tutor.
Answer the following question clearly and concisely, in 2-4 sentences.

Question: {question}
{context_block}

Answer: """
```

Step 12 — Write the generation loop

Call each of the 5 models on every question via the Ollama Python client, storing:

- model name
- generated answer
- latency (seconds)
- token count

Step 13 — Sanity check

Before running the full batch, manually read 5–10 outputs per model to catch prompt or parsing bugs early — this saves hours of wasted compute.

Step 14 — Run full generation

Run all 300 questions × 5 models = 1,500 generations. Save raw results to `raw_answers.json`.

Phase 4: Metric Implementation (Week 4)

Step 15 — Metric 1: ROUGE-L (lexical overlap)

Library: <https://github.com/google-research/google-research/tree/master/rouge>

```
python

from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=True)

def compute_rouge(reference, generated):
    scores = scorer.score(reference, generated)
    return scores['rougeL'].fmeasure
```

Step 16 — Metric 2: BERTScore (semantic similarity)

Library: https://github.com/Tiiiger/bert_score

```
python

from bert_score import score as bert_score

def compute_bertscore(references, generated_list):
    P, R, F1 = bert_score(generated_list, references, lang="en", verbose=False)
    return F1.tolist()
```

Step 17 — Metric 3: Local LLM-as-Judge (correctness/quality)

Use your largest local model (Mistral-7B) as an impartial grader, scoring 1-5.

```
python
```

```

JUDGE_MODEL = "mistral:7b"
JUDGE_PROMPT = """You are grading a student-facing AI tutor's answer.

Question: {question}
Reference (correct) answer: {reference}
AI-generated answer: {generated}

Rate the AI-generated answer from 1 to 5 for factual correctness and
completeness relative to the reference answer.
Respond with ONLY a single integer from 1 to 5."""

def llm_judge_score(question, reference, generated):
    prompt = JUDGE_PROMPT.format(question=question, reference=reference, generated=generated)
    response = ollama.chat(model=JUDGE_MODEL, messages=[{"role": "user", "content": prompt}])
    text = response['message']['content'].strip()
    try:
        return int(''.join(filter(str.isdigit, text))[:1])
    except:
        return None # flag for manual review

```

Important validity note: Since Mistral-7B is both a judge and one of the compared models, explicitly exclude its own answers from judge-based bias analysis, and validate the judge with human ratings (Step 20).

Step 18 — Run all three metrics

Score every (question, model-answer) pair and save to `scored_results.csv`.

Phase 5: Human Validation & Aggregation (Week 5)

Step 19 — Manual scoring

Have a human (you) score 50 random samples on the same 1-5 scale used by the LLM-judge.

Step 20 — Compute inter-rater agreement

Use Cohen's kappa (available in `scikit-learn` or `statsmodels`) to check how well the LLM-judge agrees with human judgment. Report this number in your Limitations section.

Step 21 — Build the leaderboard

python

```

import pandas as pd

df = pd.DataFrame(results) # rouge, bertscore, judge columns per row

leaderboard = df.groupby("model").agg(
    avg_rouge=("rouge_l", "mean"),
    avg_bertscore=("bertscore_f1", "mean"),
    avg_judge=("judge_score", "mean"),
    avg_latency=("latency_sec", "mean")
).reset_index()

for col in ["avg_rouge", "avg_bertscore", "avg_judge"]:
    leaderboard[col + "_norm"] = (leaderboard[col] - leaderboard[col].min()) / \
        (leaderboard[col].max() - leaderboard[col].min())

leaderboard["combined_score"] = leaderboard[
    ["avg_rouge_norm", "avg_bertscore_norm", "avg_judge_norm"]
].mean(axis=1)

leaderboard = leaderboard.sort_values("combined_score", ascending=False)

```

Step 22 — Subject-wise breakdown

```

python

subject_leaderboard = df.groupby(["subject", "model"]).agg(
    avg_rouge=("rouge_l", "mean"),
    avg_bertscore=("bertscore_f1", "mean"),
    avg_judge=("judge_score", "mean")
).reset_index()

```

Step 23 — Statistical significance testing

```

python

from scipy.stats import wilcoxon

model_a_scores = df[df.model == "mistral:7b"]["judge_score"]
model_b_scores = df[df.model == "phi3:mini"]["judge_score"]
stat, p_value = wilcoxon(model_a_scores, model_b_scores)
print(f"p-value: {p_value}") # p < 0.05 => statistically significant

```

Run this pairwise between your top 2-3 models to confirm differences aren't due to chance.

Phase 6: Analysis & Visualization (Week 6)

Step 24 — Overall leaderboard bar chart

```
python

import matplotlib.pyplot as plt
import seaborn as sns

sns.barplot(data=leaderboard, x="model", y="combined_score")
plt.title("Overall Model Ranking (Combined Score)")
plt.savefig("leaderboard.png", dpi=200, bbox_inches="tight")
```

Step 25 — Subject-wise heatmap

```
python

pivot = subject_leaderboard.pivot(index="model", columns="subject", values="avg_ju
sns.heatmap(pivot, annot=True, cmap="YlGnBu")
plt.title("Judge Score by Subject and Model")
plt.savefig("subject_heatmap.png", dpi=200, bbox_inches="tight")
```

Step 26 — Radar chart

Build a radar/spider chart comparing ROUGE-L, BERTScore, and Judge score across all 5 models — this is a strong visual for the paper's Results section.

Step 27 — Efficiency-frontier plot

Plot `combined_score` (y-axis) against `avg_latency` (x-axis) to identify the best accuracy-vs-speed trade-off model.

Step 28 — Qualitative error analysis

Manually read a sample of your lowest-scoring answers and categorize failure modes: hallucination, incompleteness, off-topic, formatting issues.

Phase 7: Optional Dashboard (Week 7 — optional but recommended)

Step 29 — Build an interactive leaderboard

Use Streamlit (<https://streamlit.io> — docs: <https://docs.streamlit.io>) or Gradio (<https://www.gradio.app>) to let anyone explore your results interactively (filter by subject,

model, metric).

Step 30 — Polish figures

Clean up axis labels, titles, and color schemes on all charts for the final paper.

Phase 8: Optional Extensions (For Stronger Papers / Bonus Marks)

Pick 1-2 if you have time:

1. **Prompt Sensitivity Study** — rerun generation with 3 prompt styles (direct, chain-of-thought, few-shot); see which model is most prompt-robust.
 2. **Cost/Latency vs. Accuracy Trade-off** — already partly covered in Step 27; expand into a dedicated section.
 3. **Hallucination Rate Analysis** — explicitly count factually unsupported statements per model.
 4. **Model-Size vs. Performance Correlation** — plot parameter count (2B-7B) against combined score to test "bigger is better."
 5. **Cross-Domain Robustness** — compare each model's rank on SciQ vs. RACE to test subject generalization.
-

Phase 9: Paper Writing (Week 8)

Recommended structure:

1. **Abstract** (150-200 words) — problem, method, key finding
 2. **Introduction** — motivation, gap in existing benchmarks, contributions
 3. **Related Work** — MMLU, HELM, BIG-bench, educational NLP evaluation, LLM-as-judge literature
 4. **Dataset** — SciQ + RACE description, sampling strategy, stats table
 5. **Methodology** — models, prompt template, pipeline, metric justification
 6. **Experimental Setup** — hardware, software versions, seeds, sample sizes (reproducibility)
 7. **Results** — leaderboard tables, charts, significance tests
 8. **Discussion** — accuracy vs. verbosity vs. latency trade-offs; what each metric reveals that a single metric would miss
 9. **Limitations** — sample size, judge-model bias, English-only, subject coverage
 10. **Conclusion & Future Work**
 11. **References**
-

Final Deliverables Checklist

- ☐ `dataset_sample.json` — cleaned, sampled dataset
 - ☐ `raw_answers.json` — raw model outputs
 - ☐ `scored_results.csv` — all three metrics per answer
 - ☐ Leaderboard tables (overall + subject-wise)
 - ☐ At least 3 visualizations (bar chart, heatmap, efficiency-frontier plot)
 - ☐ Human-validation sample + Cohen's kappa agreement score
 - ☐ GitHub repository with code, README, reproduction instructions
 - ☐ Final paper (PDF, formatted to target venue)
 - ☐ (Optional) Interactive Streamlit/Gradio leaderboard demo
-

Full Reference Link List

Tool/Resource	Link
Ollama	https://ollama.com
Ollama downloads	https://ollama.com/download
Hugging Face Datasets docs	https://huggingface.co/docs/datasets
Hugging Face Transformers docs	https://huggingface.co/docs/transformers
ROUGE Score library	https://github.com/google-research/google-research/tree/master/rouge
BERTScore library	https://github.com/Tiiiger/bert_score
SciQ dataset	https://huggingface.co/datasets/allenai/sciq
RACE dataset	https://huggingface.co/datasets/race
OpenBookQA dataset	https://huggingface.co/datasets/openbookqa
ARC dataset	https://huggingface.co/datasets/ai2_arc
SQuAD v1.1 dataset	https://huggingface.co/datasets/squad
Google Colab	https://colab.research.google.com
Kaggle Notebooks	https://www.kaggle.com/code
Streamlit	https://streamlit.io
Streamlit docs	https://docs.streamlit.io
Gradio	https://www.gradio.app
Python downloads	https://www.python.org/downloads/
arXiv (for literature review)	https://arxiv.org

Follow the phases in order — each one produces an artifact the next phase depends on. If you get stuck on any single step (e.g., the generation loop, the judge calibration, or the significance

testing), come back and I can write the full working code for that piece.